Pipeline Observability Console: spec

Status: draft v2 (in-house build) · Audience: Codex and the data team · Source of truth for this repo

1. What we're building

This is an in-house observability console for our ETL pipeline: MySQL and MongoDB → Airbyte → Airflow → warehouse. We build and own every part of it. That means the collectors that capture data from Airbyte, Airflow and the warehouse, the observability store, the backend API, the web app, the alert engine, and tenant onboarding. It uses no third-party dashboard tool.

On one screen it answers these questions:

- Which tenants are healthy?
- What failed, and where: Airflow, Airbyte or the warehouse?
- Why did it fail?
- How long do runs take?
- How fresh is each tenant's data?
- What changed?

Anyone can use it without PAM access. Operators can also act from it: retry runs, acknowledge alerts, pause tenants, and onboard new clients.

Phase 1 runs fully on a laptop with realistic demo data, so the CEO demo doesn't depend on prod access. Switching to live data should only mean changing environment variables.

Working name: **Observability Console**. Rename it freely; the name appears only in `frontend/src/config.ts` and the page title.

2. Current pipeline and assumptions

```
flowchart LR
  M[(MySQL per tenant)] --> AB[Airbyte connections]
  G[(MongoDB per tenant)] --> AB
  AF[Airflow: 4 hourly DAGs<br/>MySQL initial · MySQL CDC<br/>Mongo initial · Mongo CDC] -- triggers s
  AB --> W[(Warehouse)]
```

Confirm these before going live. Demo mode doesn't depend on any of them.

- Each tenant has its own Airbyte connection(s), triggered by the Airflow tasks (`AirbyteTriggerSyncOperator` or an HTTP call). That makes the Airbyte `connection_id` the cleanest way to map a run to a tenant. If tenants share connections, the tenant has to come from the Airflow task id or the stream namespace instead (§6.1, step 4).
- The DAG ids. This spec uses the placeholders `mysql_initial_load` , `mysql_cdc` , `mongo_initial_load` and `mongo_cdc` ; the real ids go in config.
- Versions: Airbyte (OSS on abctl/Helm, or Cloud), Airflow (2.x uses REST `/api/v1` ; 3.x uses `/api/v2` with token auth), and the warehouse engine.

- The alert channel (Slack, Teams or email) and the SSO provider, which is needed later.

3. What it captures

Source	Captured
Airbyte	Every sync job: status, duration, attempts, rows and bytes synced, per-stream record counts (where the version exposes them), failure summaries (origin, type, message), connection config (sync mode, enabled or disabled), server health
Airflow	DAG runs and task instances: state, queued time, start and end, try number, duration. Also: queue delay (start minus queued), task logs, the Airbyte job id from XCom, DAG paused state, DAG import errors, scheduler and metadata DB health, pool usage
Warehouse	Per tenant table: row count, last load time (`_airbyte_extracted_at`), volume change since the previous check, column snapshots for schema drift detection
Platform	Collector heartbeat and poll duration, API errors per source, alert engine heartbeat, obs DB size
Derived	Success rate, avg/p50/p95 duration, queue delay, MTTR, SLA compliance %, error groups and recurrence, tenant health state (healthy / degraded / down)

4. Architecture

```
flowchart LR
  AB[Airbyte API] --> C
  AF[Airflow REST API] --> C
  WH[(Warehouse)] --> C
  C[collector<br/>Python, every 2 min] --> DB[(obs Postgres)]
  C -- NOTIFY run_events --> DB
  DB --> API[api<br/>FastAPI]
  API -- SSE live events --> WEB[web console<br/>React]
  WEB --> API
  AL[alerter<br/>Python, every 1 min] --> DB
  AL --> N[Slack / Teams / email]
  API -- onboarding, retry --> AF
  API -- trigger sync --> AB
```

Component	Tech	Job
`collector`	Python 3.11, httpx, psycopg 3, APScheduler	Polls Airbyte, Airflow and the warehouse; links, classifies, redacts, and upserts into `obs` ; sends `NOTIFY run_events`
`api`	FastAPI, psycopg 3, sse-starlette	Read API for the console, operator actions, auth, and the SSE stream (it `LISTEN` s on `run_events`)
`alerter`	Python 3.11	Evaluates alert rules, manages the alert lifecycle, and sends notifications
`web`	React + TypeScript + Vite, Tailwind, TanStack Query, TanStack Table, Apache ECharts,	The console UI, served by nginx, which also proxies

	React Router	`/api`
`db`	Postgres 16	The observability store (schema `obs`)

Decisions that matter:

- **Only `collector` and `api` hold credentials for Airbyte, Airflow and the warehouse.** The browser never talks to those systems.
- **All collector writes are upserts keyed by `run_key`.** Restarts and re-polls never duplicate rows.
- **The collector and alerter run outside Airflow,** so the console still reports when Airflow itself is down.
- **Failures are enriched when they are collected** with the error class, a fix hint, and the redacted log. Viewers can see why a run failed without Airflow or Airbyte access.
- **Live updates flow collector → Postgres `NOTIFY` → api → SSE → browser.** There's no extra message broker.

5. Data model (Postgres, schema `obs`)

Migrations live in `sql/migrations/NNN_name.sql` and are applied by `python -m obs.migrate`. The runner records each file in `obs.schema_migrations`, and the api container runs it on start.

001_core.sql

```
create schema if not exists obs;

create table obs.tenants (
  tenant_id            text primary key,              -- slug, e.g. 'acme_retail'
  display_name         text not null,
  status               text not null default 'onboarding'
                       check (status in ('onboarding','active','paused','offboarded')),
  has_mysql            boolean not null default true,
  has_mongo            boolean not null default true,
  airbyte_connections  jsonb not null default '{}',    -- {"mysql_initial":"<uuid>","mysql_cdc":"<uuid
  warehouse_schema     text,
  freshness_sla_min    integer not null default 120,   -- hourly CDC, so 2h without a success = stale
  owner_email          text,
  created_at           timestamptz not null default now(),
  updated_at           timestamptz not null default now()
);

create table obs.pipeline_runs (
  run_key               text primary key,  -- '<dag_id>|<dag_run_id>|<task_id>|<map_index>', or 'airby
  tenant_id             text not null references obs.tenants(tenant_id),
  source_type           text not null check (source_type in ('mysql','mongo')),
  load_type             text not null check (load_type in ('initial','cdc')),
  status                text not null check (status in ('queued','running','success','failed','cancell
  failed_stage          text check (failed_stage in ('airflow','airbyte','warehouse')),
  triggered_by          text not null default 'schedule',   -- schedule | manual | retry | onboarding
  scheduled_for         timestamptz,
  queued_at             timestamptz,
  started_at            timestamptz,
  ended_at              timestamptz,
  duration_s            integer,               -- null while running
  queue_delay_s         integer,               -- started_at - queued_at
  rows_synced           bigint,
```

```
  bytes_synced          bigint,
  dag_id                text,
  dag_run_id            text,
  task_id               text,
  map_index             integer default -1,
  try_number            integer,
  airbyte_connection_id text,
  airbyte_job_id        bigint,
  airbyte_attempts      integer,
  collected_at          timestamptz not null default now()
);
create index on obs.pipeline_runs (tenant_id, started_at desc);
create index on obs.pipeline_runs (status, started_at desc);
create index on obs.pipeline_runs (started_at desc);
create index on obs.pipeline_runs (airbyte_job_id);

create table obs.run_stream_stats (
  run_key      text not null references obs.pipeline_runs(run_key) on delete cascade,
  stream_name  text not null,
  records      bigint,
  bytes        bigint,
  primary key (run_key, stream_name)
);

create table obs.error_groups (
  fingerprint     text primary key,       -- sha1(error_class + normalized message)
  error_class     text not null,
  sample_message  text,
  first_seen      timestamptz not null,
  last_seen       timestamptz not null,
  occurrences     integer not null default 1
);

create table obs.run_errors (
  run_key        text primary key references obs.pipeline_runs(run_key) on delete cascade,
  tenant_id      text not null,
  stage          text not null,           -- airflow | airbyte | warehouse
  failure_origin text,                    -- Airbyte: source | destination | replication | ...
  error_class    text not null,
  severity       text not null default 'error',
  fingerprint    text references obs.error_groups(fingerprint),
  error_message  text,                    -- redacted, max 2 KB
  hint           text,
  created_at     timestamptz not null default now()
);

create table obs.run_logs (               -- redacted logs of failed runs; successful runs are fetched
  run_key   text primary key references obs.pipeline_runs(run_key) on delete cascade,
  source    text not null check (source in ('airflow','airbyte')),
  content   text not null,                -- capped at 512 KB, tail kept
  tsv       tsvector generated always as (to_tsvector('simple', content)) stored
);
create index on obs.run_logs using gin (tsv);
```

002_warehouse_platform.sql

```
create table obs.warehouse_snapshots (
  tenant_id       text not null,
  table_name      text not null,
  row_count       bigint,
  last_loaded_at  timestamptz,
  checked_at      timestamptz not null default now(),
  primary key (tenant_id, table_name, checked_at)
);

create table obs.schema_snapshots (
  tenant_id     text not null,
  table_name    text not null,
  columns       jsonb not null,           -- [{"name":"id","type":"bigint"}, ...]
  columns_hash  text not null,
  checked_at    timestamptz not null default now(),
  primary key (tenant_id, table_name, checked_at)
);

create table obs.platform_health (
  component   text not null,              -- airflow_scheduler | airflow_metadb | airflow_pools | airf
  status      text not null check (status in ('up','degraded','down')),
  details     jsonb,
  checked_at  timestamptz not null default now(),
  primary key (component, checked_at)
);

create table obs.collector_state (
  component        text primary key,
  watermark        timestamptz,
  last_success_at  timestamptz,
  last_poll_ms     integer,
  last_error       text,
  updated_at       timestamptz not null default now()
);

create table obs.tenant_events (          -- onboarding progress
  id         bigserial primary key,
  tenant_id  text not null,
  step       text not null,
  status     text not null check (status in ('started','ok','failed')),
  message    text,
  at         timestamptz not null default now()
);
```

003_alerts_users.sql

```
create table obs.alert_rules (
  rule_id     text primary key,
  name        text not null,
  severity    text not null check (severity in ('critical','high','medium','low')),
  params      jsonb not null default '{}',
  enabled     boolean not null default true,
  repeat_min  integer not null default 240,
  channels    text[] not null default '{}'
);
```

```
create table obs.alerts (
  id                bigserial primary key,
  rule_id           text not null references obs.alert_rules(rule_id),
  fingerprint       text not null,          -- rule_id|tenant_id|source_type|...
  tenant_id         text,
  severity          text not null,
  state             text not null check (state in ('firing','acked','resolved')),
  title             text not null,
  details           jsonb,
  first_fired_at    timestamptz not null default now(),
  last_seen_at      timestamptz not null default now(),
  last_notified_at  timestamptz,
  acked_by          text,
  acked_at          timestamptz,
  resolved_at       timestamptz
);
create unique index on obs.alerts (fingerprint) where state <> 'resolved';

create table obs.notification_channels (
  channel_id  text primary key,              -- e.g. 'slack-data-eng'
  kind        text not null check (kind in ('slack','teams','email')),
  target_env  text not null,                 -- name of the env var holding the webhook URL or address
  enabled     boolean not null default true
);

create table obs.users (
  user_id        bigserial primary key,
  email          text unique not null,
  display_name   text,
  role           text not null check (role in ('viewer','operator','admin')),
  password_hash  text,                        -- null once SSO is on
  created_at     timestamptz not null default now()
);

create table obs.audit_log (
  id       bigserial primary key,
  actor    text not null,
  action   text not null,                     -- retry_run | ack_alert | resolve_alert | pause_tenant
  target   text,
  details  jsonb,
  at       timestamptz not null default now()
);
```

`004_views.sql` (§7). `005_roles.sql` creates `obs_app` (read/write, used by the services) and `obs_ro` (read-only, for ad-hoc analysis).

Volume: 10 tenants × 4 pipelines × 24 runs a day is about 350k rows a year, so no partitioning is needed. Retention is a nightly job in the collector:

- runs: 13 months
- `run_logs` : 90 days
- snapshots and platform health: 30 days, plus daily rollups

6. Collector

6.1 Poll loop

Runs every `POLL_INTERVAL_SEC` (default 120):

1. **Airflow runs.** List DAG runs for the configured DAG ids that were updated since the watermark, with a 10-minute overlap, plus anything still running. Then list their task instances. Each task that triggers an Airbyte sync counts as one pipeline run. Identify those tasks with `SYNC_TASK_REGEX` or by operator class. Record `queued_when`, start and end, try number, and state.
2. **Link to Airbyte.** Read the task's XCom `return_value`, which holds the job id when `AirbyteTriggerSyncOperator` is used. If it's missing, take the Airbyte job on the same connection whose start time is within ±5 minutes of the task start. A linked job is never inserted separately.
3. **Airbyte jobs.** For each linked job, fetch status, duration, rows, bytes, attempts and per-stream stats. Then list jobs per tenant connection directly, so syncs started outside Airflow still appear, under the key `airbyte|<job_id>` with `triggered_by = 'manual'`.
4. **Resolve the tenant.** Look up the Airbyte `connection_id` in `obs.tenants.airbyte_connections`. If that fails, fall back in this order:
   - `TENANT_TASK_REGEX` applied to `task_id`, for example `^sync_(?P<tenant>[a-z0-9_]+)$`.
   - The `rendered_map_index` of mapped tasks.
   - The stream namespace.

   Runs that can't be resolved go into the `_unmapped` tenant so they stay visible.
5. **Merge status.**
   - The run is `failed` if either the Airflow task or the Airbyte job failed.
   - `failed_stage = 'airbyte'` when the Airbyte job failed.
   - `failed_stage = 'airflow'` when the task failed but the job succeeded or never started.
   - `failed_stage = 'warehouse'` is set only by the warehouse checks.
6. **Enrich failures.**
   - Take the Airbyte failure summary from the latest attempt.
   - Fetch the Airflow task log, and the Airbyte attempt log if the failure is on the Airbyte side.
   - Redact everything (§6.4), then classify the failure and compute its fingerprint (§6.3).
   - Upsert `run_errors`, `error_groups` and `run_logs`.
7. **Notify.** After each upsert batch, send `NOTIFY run_events, '<json>'` with `{run_key, tenant_id, status, source_type, load_type}`. The payload must stay under 8 KB.
8. **Bookkeeping.** Update the watermark, `last_success_at` and `last_poll_ms`. On an exception, write `last_error` and continue. One bad tenant or API call never stops the loop.

Every `PLATFORM_CHECK_SEC` (default 60):

- Airflow `/health` (scheduler and metadata DB), `/pools` (occupied and queued slots), and `/importErrors` (broken DAGs).
- The Airbyte server health endpoint.
- A warehouse connectivity ping.

Each check writes a row to `platform_health`.

Every `WAREHOUSE_CHECK_SEC` (default 900):

- Per active tenant table, record the row count and the last load time in `warehouse_snapshots`, and the column list and its hash in `schema_snapshots`. Make the load-time column configurable, since older Airbyte destinations use `_airbyte_emitted_at` instead of `_airbyte_extracted_at`.
- In live mode, never run `count(*)` on big tables. Use catalog statistics through one adapter per engine:
  - Snowflake: `information_schema.tables.row_count`
  - BigQuery: `__TABLES__`
  - Redshift: `svv_table_info`
  - Postgres: `pg_class.reltuples`

6.2 API references (verify against the deployed versions)

Airflow 2.x, base `/api/v1`, using a read-only Viewer service account:

- Batch DAG runs: `POST /dags/~/dagRuns/list`
- Batch task instances: `POST /dags/~/dagRuns/~/taskInstances/list`
- XCom: `GET /dags/{dag_id}/dagRuns/{run_id}/taskInstances/{task_id}/xcomEntries/return_value` (with `map_index` for mapped tasks)
- Logs: `GET /dags/{dag_id}/dagRuns/{run_id}/taskInstances/{task_id}/logs/{try_number}`
- Health and diagnostics: `GET /health`, `GET /pools`, `GET /importErrors`
- Write calls, used by the api only and with a separate token:
  - `POST /dags/{dag_id}/clearTaskInstances` (retry)
  - `POST /dags/onboard_tenant/dagRuns` (onboarding)

Airflow 3.x moves these endpoints to `/api/v2` with token auth. Put the client behind an interface so either version can be swapped in.

Airbyte:

- The public API (`/api/public/v1` on recent OSS; `api.airbyte.com/v1` on Cloud) with application credentials exchanged for a bearer token. Use it to list jobs per connection and to trigger a sync for retries (`POST /jobs` with `jobType: sync`). The `duration` field is ISO-8601 (`PT4M12S`); parse it to seconds.
- The config API `POST /api/v1/jobs/get`, which returns attempts with `failureSummary.failures[]` (`failureOrigin`, `failureType`, `externalMessage`) plus stream stats.

Tests never hit live endpoints. Record JSON fixtures from staging once and replay them.

6.3 Error classification and grouping (`error_rules.yaml`)

Rules are checked in order and the first match wins. Each rule has a `class`, a `severity`, case-insensitive regex `patterns`, and a `hint`.

```
- class: MYSQL_BINLOG_PURGED
  severity: critical
  patterns: ["binlog.*(purged|not available)", "Could not find first log file name in binary log index
  hint: CDC position lost. Needs a stream reset or initial load. Check binlog retention on the source.
- class: MONGO_CHANGE_STREAM_LOST
  severity: critical
  patterns: ["ChangeStreamHistoryLost", "resume (token|point).*(no longer|not found)", "oplog.*(rolled
  hint: Resume token fell off the oplog. Needs a resync. Consider a larger oplog.
```

```
- class: SOURCE_AUTH
  severity: error
  patterns: ["Access denied for user", "Authentication failed", "auth(entication)? fail"]
  hint: Credentials changed or expired on the client side. Contact the tenant owner.
- class: SOURCE_UNREACHABLE
  severity: error
  patterns: ["Communications link failure", "Connection refused", "No route to host", "ServerSelection
  hint: Client DB or network is down, or our IP isn't allowlisted.
- class: SCHEMA_CHANGE
  severity: warning
  patterns: ["schema (change|mismatch)", "column .* (does not exist|not found)", "breaking change"]
  hint: Source schema changed. Refresh the schema in Airbyte and review downstream models.
- class: DESTINATION_ERROR
  severity: error
  patterns: ["warehouse.*(quota|permission|denied)", "destination.*(failed|error)"]
  hint: Warehouse rejected the write. Check permissions, quota or table locks.
- class: OUT_OF_MEMORY
  severity: error
  patterns: ["OutOfMemoryError", "exit code 137", "OOMKilled"]
  hint: Sync pod ran out of memory. Raise resource limits for this connection.
- class: AIRFLOW_INFRA
  severity: error
  patterns: ["zombie", "Executor reports task instance", "SIGTERM", "SIGKILL"]
  hint: Airflow killed the task. Not a data problem. Check workers and scheduler.
- class: UNKNOWN
  severity: error
  patterns: [".*"]
  hint: Unclassified. Read the log and add a rule.
```

When Airbyte reports `failureOrigin = destination`, classify the failure as `DESTINATION_ERROR` even if no pattern matches.

Fingerprint: lowercase the message and replace numbers, UUIDs, hex ids, timestamps, IPs and quoted values with placeholders. Then compute `sha1(error_class + normalized)`. The same root cause across 30 runs becomes one error group with a count, instead of 30 rows.

`python -m obs.collector.classify --replay` re-classifies existing errors after the rules change.

6.4 Redaction (`redact.py`)

Before anything is stored or returned by the API, mask the following:

- passwords (`password=` , `pwd=`)
- bearer tokens and JWTs
- connection strings with embedded credentials (`mongodb://user:pass@` , `mysql://user:pass@`)
- AWS-style keys
- email addresses

Unit-test it with real-world-looking samples. The on-demand log endpoint redacts on the way out too.

6.5 Modes

- `MODE=demo` : seeds and simulates everything (§16). No external calls are made, and operator actions are simulated.
- `MODE=live` : polls the real systems using the env vars in `.env` .

7. Health logic and metric definitions

004_views.sql

```
create or replace view obs.v_latest_runs as
select distinct on (tenant_id, source_type, load_type) *
from obs.pipeline_runs
order by tenant_id, source_type, load_type, started_at desc nulls last;

create or replace view obs.v_freshness as
select t.tenant_id, t.display_name, s.source_type, t.freshness_sla_min,
       max(r.ended_at) as last_success_at,
       round(extract(epoch from now() - max(r.ended_at)) / 60) as minutes_stale
from obs.tenants t
cross join (values ('mysql'), ('mongo')) s(source_type)
left join obs.pipeline_runs r
  on r.tenant_id = t.tenant_id and r.source_type = s.source_type and r.status = 'success'
where t.status = 'active'
  and ((s.source_type = 'mysql' and t.has_mysql) or (s.source_type = 'mongo' and t.has_mongo))
group by t.tenant_id, t.display_name, s.source_type, t.freshness_sla_min;

create or replace view obs.v_stuck_runs as
with baseline as (
  select tenant_id, source_type, load_type,
         percentile_cont(0.5) within group (order by duration_s) as median_s
  from obs.pipeline_runs
  where status = 'success' and started_at > now() - interval '7 days'
  group by 1, 2, 3
)
select r.run_key, r.tenant_id, r.source_type, r.load_type, r.started_at,
       round(extract(epoch from now() - r.started_at) / 60) as running_min,
       round((b.median_s / 60)::numeric) as usual_min
from obs.pipeline_runs r
join baseline b using (tenant_id, source_type, load_type)
where r.status = 'running'
  and extract(epoch from now() - r.started_at) > 3 * b.median_s;

create or replace view obs.v_tenant_health as
select t.tenant_id, t.display_name, t.status as tenant_status, t.freshness_sla_min,
       coalesce(r.failures_24h, 0) as failures_24h,
       f.worst_stale_min,
       case
         when t.status <> 'active' then t.status
         when exists (select 1 from obs.v_latest_runs l
                      where l.tenant_id = t.tenant_id and l.load_type = 'cdc' and l.status = 'failed')
              or f.never_succeeded
              or f.worst_stale_min > t.freshness_sla_min then 'down'
         when coalesce(r.failures_24h, 0) > 0
              or f.worst_stale_min > 0.75 * t.freshness_sla_min
              or exists (select 1 from obs.v_stuck_runs s where s.tenant_id = t.tenant_id) then 'degra
         else 'healthy'
       end as health
from obs.tenants t
left join (select tenant_id, max(minutes_stale) as worst_stale_min,
                  bool_or(last_success_at is null) as never_succeeded
           from obs.v_freshness group by tenant_id) f using (tenant_id)
```

```
left join (select tenant_id, count(*) filter (where status = 'failed') as failures_24h
           from obs.pipeline_runs where started_at > now() - interval '24 hours'
           group by tenant_id) r using (tenant_id);
```

Metric definitions. Use these exact definitions everywhere so the numbers never disagree between pages.

- **Success rate:** success ÷ (success + failed) in the range. Cancelled and skipped runs are excluded.
- **Duration avg / p50 / p95:** successful runs only, in minutes, split by `source_type` + `load_type`. Use `percentile_cont(...)::numeric` before rounding.
- **Queue delay:** avg and p95 of `queue_delay_s`. When this rises, Airflow capacity is the problem, not the data.
- **MTTR:** for each failure, the time until the next success of the same tenant, source and load type. Report the average over the range.
- **SLA compliance %:** the share of 5-minute buckets in the range where the tenant source's `minutes_stale` was within its SLA. Compute it from `pipeline_runs` success times, not from snapshots.
- **Volume anomaly:** a table's row count dropped more than 30% from the previous snapshot, or grew more than 5× the 7-day average delta.
- **Schema drift:** `columns_hash` changed from the previous snapshot. Store the column diff (added, removed, type changed).

8. Backend API (`/api` , JSON, OpenAPI at `/api/docs`)

Method and path	Role	Returns
`POST /auth/login` , `POST /auth/logout` , `GET /me`	any	Session (JWT in an httpOnly cookie), current user and role
`GET /overview?range=24h`	viewer	KPIs, health counts, platform strip, runs-per-hour series, duration by pipeline, recent failures
`GET /tenants`	viewer	Tenant list with health, 7-day SLA compliance, failures, avg duration, last success
`GET /tenants/{id}`	viewer	Detail: config, health, latest runs, freshness per table, open alerts, onboarding events
`GET /tenants/{id}/timeline?range=`	viewer	Runs per pipeline lane for the swimlane chart

`PATCH /tenants/{id}`	admin	Pause or resume, SLA, owner
`GET /runs?tenant=&status=&source=&load=&stage=&error_class=&from=&to=&q=&page=&sort=`	viewer	Paginated runs. `format=csv` exports
`GET /runs/{run_key}`	viewer	Full run: Airflow and Airbyte details, attempts, stream stats, error, the previous 10 runs of the same pipeline
`GET /runs/{run_key}/log`	viewer	Redacted log (stored copy, or fetched live for successful runs)
`POST /runs/{run_key}/retry`	operator	Clears the Airflow task, or triggers an Airbyte sync. Audited
`GET /errors/groups?range=`	viewer	Error groups with counts, tenants affected, first and last seen, daily trend
`GET /errors/groups/{fingerprint}`	viewer	The group plus its runs
`GET /metrics/durations?group_by=pipeline|tenant&range=`	viewer	avg / p50 / p95
`GET /metrics/mttr?range=`, `GET /metrics/failures-heatmap?range=`	viewer	MTTR by tenant; failures by hour of day × weekday
`GET /freshness`, `GET /volume-anomalies`, `GET /schema-changes`	viewer	Warehouse views
`GET /alerts?state=`, `POST /alerts/{id}/ack`, `POST /alerts/{id}/resolve`	viewer / operator	Alerts and their lifecycle. Audited
`GET /alert-rules`, `PUT /alert-rules/{id}`, `GET /channels`, `POST /channels/{id}/test`	admin	Rule parameters and channels; sends a test notification
`POST /onboarding/preflight`	operator	Dry-run connectivity and CDC checks without creating anything

`POST /onboarding`, `GET /onboarding/{tenant_id}`	operator	Starts onboarding; returns step progress
`GET /platform/health`	viewer	Component status, pools, import errors, collector stats
`GET /search?q=`	viewer	Full-text search over error messages and stored logs
`GET /audit`	admin	Audit log
`GET /stream`	viewer	SSE events: `run.updated`, `alert.fired`, `alert.resolved`, `onboarding.step`, `platform.changed`

Rules for the API:

- All list endpoints paginate on the server.
- All times are returned in ISO-8601 UTC.
- Errors use one shape: `{error: {code, message}}`.
- Generate frontend types from the OpenAPI schema (`openapi-typescript`) so the UI and the API never drift.

9. Web console

Stack: React + TypeScript (strict), Vite, Tailwind, TanStack Query (with SSE events invalidating queries, plus 30s polling as a fallback), TanStack Table, Apache ECharts, React Router. It supports light and dark mode.

Design rules:

- Use the same status colors everywhere:
  - healthy / success: green
  - degraded: amber
  - down / failed: red
  - running: blue
  - onboarding / paused / not started: gray
- Every view has a shareable URL with its filters and time range in the query string.
- Times show in the viewer's local time zone, with UTC on hover.
- The global time range picker offers 1h / 6h / 24h / 7d / 30d / custom.
- A global search box (errors and logs) sits in the header.
- TV mode (`?tv=1`) hides navigation and enlarges the overview, for the office screen and the demo.

Pages:

1. **Overview (home).**
   - KPI cards: healthy / degraded / down tenants, success rate, failed runs, CDC avg and p95, SLA compliance, running now, open alerts.
   - Platform strip: Airflow scheduler, Airbyte, warehouse, collector, each with a status dot.
   - Tenant health grid: one row per tenant; columns for the latest status of MySQL CDC, MySQL initial, Mongo CDC and Mongo initial, plus freshness and last run. Problems sort to the top.
   - Charts: runs per hour stacked by status; duration trend by pipeline.
   - Recent failures list.
   - Live activity feed, fed by SSE.
2. **Tenants.** A searchable list with health, SLA compliance, failures, average duration and last success.
3. **Tenant detail.**
   - Header: health badge, status, owner, schema, onboarded date, and a pause/resume button.
   - Swimlane timeline: one lane per pipeline, one block per run, colored by status. Clicking a block opens the run.
   - Charts: duration against the 7-day median; rows synced, with a per-stream breakdown.
   - Lists: freshness per table, schema changes, errors, open alerts.
   - Connections and config; onboarding steps if the tenant is still onboarding.
4. **Runs explorer.** A filterable table with CSV export.
5. **Run detail.**
   - Summary: times, queue delay, duration against the usual, rows, bytes, attempts.
   - An Airflow section and an Airbyte section, the latter with attempts and failure origin and type.
   - The error with its fix hint.
   - Log viewer: line numbers, in-log search, highlighted error lines, and a download of the redacted copy.
   - A retry button (operator), and the previous 10 runs of the same pipeline.
6. **Errors.**
   - Error groups: class, sample message, occurrence count, tenants affected, first and last seen, sparkline. A group opens its runs.
   - Charts: failures by stage, by class and by tenant; a failures heatmap by hour × weekday; MTTR by tenant.
7. **Freshness and volume.**
   - Minutes stale against SLA per tenant and source.
   - SLA compliance over 7 and 30 days.
   - Volume anomalies, and schema drift with column diffs.
8. **Alerts.**
   - Firing, acknowledged and resolved alerts, with ack and resolve actions and history.
   - Rule settings (admin) and notification channels with "send test".
9. **Onboard tenant.** A four-step wizard (tenant details → sources → pre-flight results → review and start), then a live progress page.
10. **Platform.** Component status history, Airflow pools, DAG import errors, collector poll stats, and obs DB size.
11. **Admin.** Users and roles, and the audit log.

10. Alert engine (`alerter`)

Rules are built-in types implemented in code. Admins edit only parameters, severity, channels and on/off in the UI; no free-form SQL is stored. The alerter evaluates all rules every 60 seconds.

rule_id	Fires when	Default params	Severity
`cdc_consecutive_failures`	The last N CDC runs for a tenant + source failed	`n: 2`	high
`cdc_position_lost`	Any `MYSQL_BINLOG_PURGED` or `MONGO_CHANGE_STREAM_LOST`	none	critical
`freshness_breach`	`minutes_stale > freshness_sla_min × pct/100`	`pct: 100`	high
`stuck_run`	Running longer than multiplier × the 7-day median	`multiplier: 3`	medium
`volume_anomaly`	Row-count drop or spike (§7)	`drop_pct: 30`	medium
`schema_drift`	Columns changed on a tenant table	none	low
`queue_backlog`	p95 queue delay over 15 min in the last hour	`minutes: 15`	medium
`dag_import_errors`	Airflow reports any import errors	none	high
`platform_down`	Any platform component `down` for 2 consecutive checks	none	critical
`collector_silent`	The collector's `last_success_at` is older than N minutes	`minutes: 10`	critical

Lifecycle:

- The fingerprint is `rule_id|tenant_id|source_type` , and there is one open alert per fingerprint.
- A new condition creates a `firing` alert and notifies immediately.
- While the condition still holds, the alerter updates `last_seen_at` and re-notifies every `repeat_min` . It skips the re-notification if the alert is `acked` .
- When the condition clears, the alert is `resolved` and a resolve notification is sent.

Notifications go out through Slack incoming webhooks, Teams Workflows webhooks, or SMTP email. Each message carries the title, tenant, severity, short reason, fix hint, and a deep link to the console page. The alerter writes its own heartbeat to `platform_health` .

11. Operator actions

Every action writes to `audit_log` and is disabled in the UI for roles that can't use it.

- **Retry run:**
  - Airflow-triggered runs: clear the failed task instance (`only_failed: true`).
  - Airbyte-only runs: trigger a sync on the same connection.

The new attempt appears live with `triggered_by = 'retry'`.

- **Acknowledge or resolve an alert.**
- **Pause or resume a tenant:** sets `obs.tenants.status`. Registry-driven DAGs (§12) skip paused tenants.
- **Onboard a tenant** (§12).

In demo mode, every action is simulated end to end. A retry produces a new run that succeeds after about 30 seconds.

12. Tenant onboarding

The flow:

1. **Wizard input.**
   - Tenant: slug and display name.
   - MySQL: host, port, database, user and a secret reference.
   - Mongo: connection secret reference and database.
   - Warehouse: schema (defaults to the slug).
   - Freshness SLA, owner email, and "run initial load now".

   Passwords are never typed into the console. It takes secret manager paths (Vault, AWS Secrets Manager or similar).

2. **Pre-flight** (`POST /onboarding/preflight`, shown as pass/fail with fixes):
   - MySQL: `binlog_format = ROW`, `binlog_row_image = FULL`, binlog retention long enough to survive a weekend outage, and the user has `REPLICATION SLAVE`, `REPLICATION CLIENT` and `SELECT`.
   - Mongo: it's a replica set or sharded cluster (change streams need one), and the user can read the DB and open change streams.

3. **Start.** The api inserts the tenant with status `onboarding` and triggers the Airflow DAG `onboard_tenant` with the wizard data as `conf`.

4. **The `onboard_tenant` DAG** (shipped in `airflow/dags/`) runs these steps, writing a `tenant_events` row for each:

   1. Create the Airbyte sources and run their connection checks.
   2. Create the connections (initial and CDC) with namespace = warehouse schema.
   3. Save the connection ids to `obs.tenants`.
   4. Trigger the initial loads.
   5. Set status `active`.

   The console streams each step live.

Prerequisite in the existing DAG repo, which the team needs to review: the four hourly DAGs should stop hard-coding tenants. Each one should start with a `get_active_tenants` task that reads `obs.tenants` (status `active`), then `.expand()` the sync task over the result (dynamic task mapping at runtime). Set `map_index_template` (Airflow 2.9+) so each mapped task shows the tenant id. After that, a new or resumed tenant is picked up on the next hourly run with no deploy. The DAG must not query the DB at parse time. This repo ships `airflow/dags/README.md` with the exact change; it does not edit the team's DAGs.

13. Security

- **Roles.** Viewer for the wider company, Operator for the data team, Admin for two people. The demo uses local users with bcrypt passwords. Later phases use SSO through OIDC (Google, Azure AD or Okta), mapping groups to roles.
- **Service credentials.**
  - The collector uses read-only accounts: Airflow Viewer, read-only Airbyte, and a warehouse role limited to the catalog and tenant schemas.
  - The api's write token can only clear task instances on the four DAGs and trigger `onboard_tenant`.
- **Network.** The obs DB is not exposed. Only the web container is published, and it proxies `/api`.
- **Application hardening.**
  - Everything shown or stored is redacted.
  - Cookies are httpOnly, Secure and SameSite=Lax.
  - Mutating endpoints are CSRF-protected.
  - Login is rate-limited.
  - Admin changes are audited.

14. Repo layout

```
obs-console/
├── AGENTS.md
├── SPEC.md
├── docker-compose.yml
├── Makefile
├── .env.example
├── sql/migrations/                001_core.sql … 005_roles.sql
├── backend/
│   ├── Dockerfile
│   ├── pyproject.toml
│   └── obs/
│       ├── settings.py  db.py  migrate.py  auth.py  audit.py  redact.py
│       ├── api/                   main.py, deps.py, schemas.py, routers/{auth,overview,tenants,runs,e
│       ├── collector/             __main__.py, airflow.py, airbyte.py, linker.py, classify.py, finger
│       ├── alerter/               __main__.py, rules.py, notify.py
│       ├── actions/               retry.py, tenants.py
│       ├── onboarding/            preflight.py, service.py
│       └── demo/                  seed.py, ticker.py, scenarios.py, simulate_actions.py
├── frontend/
│   ├── Dockerfile                 build → nginx (serves SPA, proxies /api, SSE-safe)
│   ├── nginx.conf
│   ├── package.json  vite.config.ts  tailwind.config.ts
│   └── src/
│       ├── config.ts  main.tsx  router.tsx
│       ├── api/                   generated types + client + useLiveEvents.ts
│       ├── components/            KpiCard, StatusPill, HealthGrid, Swimlane, DurationChart, RunsTable
│       └── pages/                 Login, Overview, Tenants, TenantDetail, Runs, RunDetail, Errors, Fr
├── airflow/dags/                  onboard_tenant.py, tenant_registry.py, README.md
└── tests/
    ├── backend/                   fixtures/ (recorded API JSON), test_linker.py, test_classify.py, te
    └── e2e/                       demo.spec.ts (Playwright)
```

15. Local run

`docker-compose.yml` (skeleton; keep images pinned):

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: obs
      POSTGRES_USER: obs
      POSTGRES_PASSWORD: ${OBS_DB_PASSWORD:-obs_local}
    volumes:
      - obs-db:/var/lib/postgresql/data
    ports: ["5433:5432"]
  api:
    build: ./backend
    command: sh -c "python -m obs.migrate && uvicorn obs.api.main:app --host 0.0.0.0 --port 8000"
    env_file: .env
    environment: { MODE: "${MODE:-demo}" }
    depends_on: [db]
  collector:
    build: ./backend
    command: python -m obs.collector
    env_file: .env
    environment: { MODE: "${MODE:-demo}" }
    depends_on: [api]
  alerter:
    build: ./backend
    command: python -m obs.alerter
    env_file: .env
    environment: { MODE: "${MODE:-demo}" }
    depends_on: [api]
  web:
    build: ./frontend
    ports: ["8080:80"]
    depends_on: [api]
volumes:
  obs-db:
```

In `nginx.conf`, proxy `/api/` to `api:8000`. For `/api/stream`, set `proxy_buffering off`, `proxy_read_timeout 1h` and `proxy_http_version 1.1`; otherwise SSE events arrive in bursts or not at all.

Makefile targets:

- `up`
- `demo` (MODE=demo, up, seed, ticker on)
- `live`
- `down`
- `reset` (drops volumes)
- `seed`
- `test` (backend + frontend unit)
- `e2e` (Playwright against `make demo`)
- `lint`

- `types` (regenerate the TS types from OpenAPI)

Demo login: `admin@demo.local`, `operator@demo.local` and `viewer@demo.local`, with the password from `DEMO_PASSWORD` (default `demo`). The console is at http://localhost:8080; TV mode is at `/?tv=1`.

16. Demo data

`demo/seed.py` creates 10 fictional tenants with 7 days of history. `demo/ticker.py` then adds a new batch of runs every ~60 seconds, compressing "hourly" into minutes so the console visibly moves. It sends `NOTIFY` exactly like the live collector does.

Realism rules:

- Durations are log-normal per tenant: small tenants run CDC in 1–5 minutes, big ones in 10–25. Initial loads take 3–6× longer.
- Queue delay is mostly under 1 minute, with an evening spike.
- About 4% of runs fail, with realistic redacted messages and log tails from §6.3. Rows track duration, and per-stream stats cover 5–15 tables per tenant.
- Warehouse and schema snapshots are written every 15 simulated minutes, and platform health every minute.

Scripted scenarios that must be present every time:

- `acme_retail`: MySQL CDC failing since 02:00 with `MYSQL_BINLOG_PURGED`. Down, past its SLA, critical alert firing.
- `bluefin_health`: Mongo CDC failed with `SOURCE_UNREACHABLE` on 3 attempts. Degraded and near its SLA.
- `northwind`: one MySQL CDC run currently stuck at about 3× its usual duration.
- `orbit_logistics`: a 40% row-count drop on one table yesterday, plus one schema drift (a column added).
- `cedar_foods`: failures that cluster at night, which shows up in the heatmap.
- One Airflow import error that appeared and was fixed yesterday, visible in Platform history.
- Onboarding `zenith_pay` from the wizard during the demo walks through its steps in about 60 seconds and turns green.
- Retrying acme_retail's failed run from the run page shows a new run that goes running → success. The tenant stays down until the simulated resync, which shows that the binlog problem needs a resync, not a retry.

17. Build plan for Codex

Work through the milestones in order. The demo is ready after M4; stop there for review.

Milestone	Scope	Done when
M0 Scaffold	Repo layout, compose, Makefile, `.env.example`, migration runner, all migrations, api `/health`, frontend shell with nav, login page and theme	`make up` serves the empty console at :8080 and you can log in
M1 Demo data	seed, ticker, all §16 scenarios, `NOTIFY` events	5–7k runs exist, every scenario is queryable, and `make reset && make demo` is reproducible
M2 API	Every read endpoint in §8, plus SSE, search, auth and roles, with tests	OpenAPI is complete, TS types are generated, and the API tests pass

M3 Core UI	Overview, Tenants, Tenant detail (with swimlane), Runs explorer, Run detail (with log viewer), live updates	You can click from a red tenant on the overview to its failed run's log; the overview updates live without a reload
M4 Rest of demo	Errors, Freshness and volume, Alerts (engine + page + test send), Onboarding wizard (simulated), Platform, Admin, retry and pause (simulated), TV mode, Playwright smoke test	The full §18 walkthrough works offline on a laptop and `make e2e` passes
M5 Live collection	Airflow and Airbyte clients, linker, classifier, warehouse adapter, platform checks, fixture-based tests	Pointed at staging, the numbers match the Airflow UI for 20 sampled runs
M6 Live actions	Real retry, pause, the `onboard_tenant` DAG, pre-flight checks, the DAG migration README	A staging tenant is onboarded end to end from the wizard, and a real retry shows up live
M7 Hardening	OIDC SSO, retention job, backups, deployment (VM + compose, or Helm), security review	Running in the shared environment behind SSO

18. Demo walkthrough (about 6 minutes)

1. **Our own console.** Open the overview in TV mode. "Every tenant and every pipeline, live. Built in-house, and anyone can see it without production access."
2. **Something's broken.** `acme_retail` is red at the top. Open it. The swimlane shows MySQL CDC failing since 2 AM. Open the run: the error is classified as a purged binlog and the hint says it needs a resync. The redacted log is right there.
3. **Patterns across tenants.** On the Errors page, 30 failures collapse into a few groups. The heatmap shows `cedar_foods` failing at night. Show MTTR by tenant.
4. **Performance.** Show CDC avg against p95, then the stuck `northwind` run at 3× its usual. Queue delay shows whether Airflow capacity is the bottleneck.
5. **Freshness, volume and schema.** `bluefin_health` is near its 2-hour SLA. `orbit_logistics` shows a 40% row drop and a new column.
6. **Alerts.** Show the firing critical alert, acknowledge it, and send a test notification to Slack.
7. **Onboarding.** Run the wizard for `zenith_pay`, pass pre-flight, start it, and watch the steps complete live until it turns green on the overview.
8. **Next steps.** SSO, per-tenant cost, data-quality checks, and a weekly SLA email to tenant owners.

19. Open questions for the team

- Airflow and Airbyte versions; Airbyte OSS or Cloud.
- The warehouse engine, and whether the destination writes `_airbyte_extracted_at` or `_airbyte_emitted_at`.
- How tenants map to Airbyte connections and Airflow tasks today.
- Where the console will run (a VM, k8s, or next to Airflow), and who owns it.
- The alert channel and on-call expectations per severity.
- The secret manager onboarding should reference.
- The freshness SLA per tenant. Is 2 hours right for everyone?
- Who gets Operator rights (retry, pause, onboard).